

Lecture 12

벽돌깨기 게임으로 배우는 프로젝트 종합

Unity를 이용한 2D 게임프로그래밍

강영민

동명대학교 게임공학과

2026년 1학기

목차

- 1 학습목표
- 2 왜 벽돌깨기인가?
- 3 게임 객체 하이어라키
- 4 Unity 2D 물리 엔진
- 5 전체 게임 기획
- 6 패들과 볼 구현
- 7 벽돌과 충돌 감지
- 8 프리팹 (Prefab)
- 9 스크립터블 객체 (ScriptableObject)
- 10 GameManager와 점수/생명 시스템
- 11 HUD 구현
- 12 WebGL 빌드와 깃헙 배포
- 13 구현 순서와 도전 과제
- 14 전체 아키텍처 정리

학습목표

이번 강의에서 배울 내용

- **벽돌깨기(Brick Breaker)** 게임을 만들며 **한 학기 동안 배운 요소를 종합한다.**
- 게임 객체 **하이어라키**와 **프리팍**으로 재사용 가능한 구조를 설계한다.
- **스크립터블 객체**로 데이터(레벨, 설정)와 로직을 분리한다.
- **물리 엔진(Rigidbody2D)**과 **충돌 이벤트**로 게임 플레이를 구현한다.
- **HUD**로 점수/생명/레벨을 표시하고, **WebGL 빌드**로 깃헙에 배포한다.

완성 목표

패들 · 볼 · 벽돌의 물리 충돌, 점수/생명 시스템, 게임 오버 UI를 갖추고
브라우저에서 실행되는 벽돌깨기 게임을 깃헙 Pages로 배포한다.

왜 벽돌깨기인가? - 교육 목표와의 일치

벽돌깨기는 종합 실습으로 완벽한 선택

한 학기 동안 따로따로 배운 개념들이 하나의 게임 안에서 자연스럽게 연결된다. 난이도가 적절하고, 모든 핵심 기술을 빠짐없이 다룰 수 있다.

요소	벽돌깨기에서의 적용	난이도
게임 객체 하이어라키	Canvas > HUD, Bricks(부모), Paddle, Ball 구조화	★
프리팸	벽돌, 불, 파워업 등 재사용 객체	★
스크립터블 객체	레벨 데이터(벽돌 배치), 게임 설정(속도/난이도)	★★
키보드/마우스 입력	패들 이동, 불 발사, 일시정지, UI 클릭	★
HUD 구현	점수, 생명, 현재 레벨, 게임 오버 UI	★★
WebGL 빌드	브라우저에서 실행 가능	★
깃헙 배포	GitHub Pages 호스팅, 코드 버전 관리	★

학습 효과가 높은 이유

다루게 되는 핵심 기술

- 물리 엔진 – Rigidbody2D, Collider2D
- 이벤트 시스템 – OnCollisionEnter2D, OnTriggerEnter2D
- UI와 게임 로직의 분리 – HUD / GameManager
- 데이터 주도 설계 – ScriptableObject
- 실제 프로젝트 워크플로우 – 빌드 & 배포

순차적 구현 로드맵

- 1 기본 패들 + 볼 물리
- 2 벽돌과 총돌 감지
- 3 게임 매니저 + 점수 시스템
- 4 HUD UI
- 5 프리팹 & ScriptableObject 리팩토링
- 6 WebGL 빌드 & 배포

결론

교육 목표와 **완전히 일치**하는 프로젝트.
막히는 부분은 언제든지 질문하세요! 🐼

[이론] 게임 오브젝트와 컴포넌트 - 구성(Composition) 모델

게임 오브젝트(GameObject)

- 씬에 존재하는 모든 것의 기본 단위
- 그 자체는 빈 컨테이너 - 항상 Transform(위치/회전/크기)만 기본 보유
- 기능은 컴포넌트를 부착하여 부여

컴포넌트(Component)

- 기능/데이터의 최소 단위 (SpriteRenderer, Rigidbody2D, Collider2D, 스크립트 등)
- 한 GameObject에 여러 개 부착 가능
- 스크립트(MonoBehaviour)도 컴포넌트의 하나

구성(Composition) > 상속(Inheritance)

복잡한 클래스 계층 대신 필요한 컴포넌트를 조합해 객체를 만든다.

예) **Ball** = GameObject
+ SpriteRenderer (보이기)
+ Rigidbody2D (물리)
+ CircleCollider2D (충돌)
+ Ball.cs (행동)

하이어라키(Hierarchy) - 부모-자식

- 자식은 부모의 상대 좌표로 배치
- 부모의 이동/회전/스케일이 자식에 전파
- 빈 오브젝트로 그룹화(예: Bricks) → 일괄 관리/활성화

씬 구조 설계 - 하이어라키

BrickBreaker (씬) 하이어라키

▽ Canvas (HUD)

ScoreText

LivesText

LevelText

GameOverPanel

Ball (프리팸)

Paddle (프리팸)

▽ Bricks (부모)

Brick (0,0) (프리팸 인스턴스)

Brick (0,1)

...

Walls (좌/우/상단 벽)

DeadZone (하단 트리거)

▽ GameManager

LevelData (S0 참조)

GameSettings (S0 참조)

구조화 원칙

- 부모 오브젝트로 묶기: 벽돌은 Bricks 빈 오브젝트의 자식으로 두어 한꺼번에 관리
- 관심사 분리: UI는 Canvas, 게임 로직은 GameManager
- 경계 처리: 벽(반사)과 DeadZone(낙하 감지)을 분리

Tag 설정

Ball, Paddle, Brick 등에 **Tag**를 부여하면 충돌 시 CompareTag()로 대상을 쉽게 구분할 수 있다.

[이론] 2D 물리 엔진의 동작 원리

물리 시뮬레이션 루프

- Unity 2D는 **Box2D** 기반 물리 엔진 사용
- 물리는 고정 시간 간격(Fixed Timestep)으로 갱신
- 물리 코드는 `FixedUpdate()`에서 처리
- 엔진이 속도·힘·충돌을 계산해 위치를 결정

Update vs FixedUpdate

Update	렌더 프레임마다(가변) – 입력 처리에 적합
FixedUpdate	고정 간격 – 물리(힘/속도)에 적합

왜 Transform 직접 이동 대신 물리?

좌표를 직접 옮기면 빠른 물체가 충돌을 관통(터널링)할 수 있다. 물리로 움직이면 충돌 반응·반사가 일관되게 처리된다.

핵심 물리량

- 속도(velocity) – 방향×빠르기
- 힘(force) – `AddForce`로 가속
- 반발(Bounciness) – 충돌 후 에너지 보존(1이면 완전 반사)
- 질량/중력 – Mass, Gravity Scale

2D 물리 컴포넌트 개요

Rigidbody2D

- 오브젝트를 물리 시뮬레이션에 등록
- **Body Type:**
 - **Dynamic:** 힘/속도로 움직임 (볼)
 - **Kinematic:** 코드로 직접 이동 (패들)
 - **Static:** 고정 (벽, 벽돌)
- 볼: **Gravity Scale = 0** (중력 제거)

Collider2D

BoxCollider2D(패들/벽돌), **CircleCollider2D**(볼)로 충돌 영역 정의

Physics Material 2D – 완전 반사

볼이 속도를 잃지 않고 튕기게 하려면 물리 머티리얼을 만든다.

- **Friction = 0** (마찰 없음)
- **Bounciness = 1** (완전 탄성)

볼의 Rigidbody2D / Collider2D에 지정.

Collision vs Trigger

Collider의 **Is Trigger**를 끄면 충돌(반사), 켜면 통과+이벤트만 발생.
볼 ↔ 벽돌은 충돌, DeadZone은 트리거.

게임 개요서 (Game Design Document)

기본 정보

제목	Brick Breaker (벽돌깨기)
장르	아케이드 / 액션
플랫폼	WebGL (브라우저)
엔진	Unity 6 (2D Core)
인원	1인 플레이
조작	마우스 / 키보드

핵심 재미 (Core Fun)

- 공의 반사 각도를 패들로 조절하는
손맛
- 벽돌이 한 번에 깨질 때의 쾌감
- 생명을 잃지 않고 레벨을 클리어하는
긴장감

한 줄 컨셉

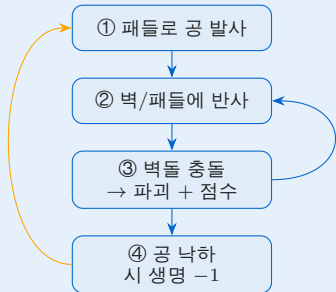
“공을 튕겨 화면의 모든 벽돌을 부숴라.”

학습 목적

물리·프리팸·스크립터블 객체·HUD·배포를 하나의 완성된 게임으로 통합 실습

핵심 게임 루프와 승패 규칙

핵심 루프 (Core Loop)



승리 조건 (Win)

현재 레벨의 모든 벽돌 파괴 ⇒ 레벨 클리어 (다음 레벨 또는 클리어 화면)

패배 조건 (Lose)

공이 패들 아래로 떨어져 생명이 모두 소진 ⇒ Game Over

세션 길이

한 레벨 약 1~3분, 짧고 반복 가능한 플레이

게임 요소 명세 (Entity Specification)

게임에 등장하는 오브젝트와 명세

요소	역할 / 행동	주요 컴포넌트
패들 (Paddle)	좌우로 움직여 공을 받아침	BoxCollider2D, Paddle.cs
볼 (Ball)	일정 속도로 이동, 부딪히면 반사	Rigidbody2D, CircleCollider2D, Physics Material 2D, Ball.cs
벽돌 (Brick)	공에 맞으면 손상/파괴, 점수 부여	SpriteRenderer, BoxCollider2D, Brick.cs (프리팹)
벽 (Wall)	좌/우/상단 경계, 공 반사	BoxCollider2D (Static)
데드존 (DeadZone)	하단에서 떨어진 공 감지	BoxCollider2D (Trigger), DeadZone.cs
파워업 (PowerUp)	(선택) 멀티볼/패들 확장 등	Collider2D, 프리팹

조작 설계 (Controls)

입력 매핑

입력	동작
마우스 이동	패들 좌우 이동
마우스 좌클릭	공 발사
A / D, ← / →	패들 이동(키보드 모드)
Esc	일시정지 (도전 과제)

설계 의도

- 직관성 - 마우스만으로 즉시 플레이
- 접근성 - 키보드 대체 입력 제공
- 패들의 **맞은 지점**에 따라 반사각이 달라지도록 설계(심화)

구현 연결

Paddle.cs의 useMouse 플래그로 마우스/키보드 모드를 전환.

점수 · 생명 · 진행 시스템 (Progression)

점수 (Score)

일반 벽돌	+10
내구도 벽돌(2HP)	+20
파괴 시 호출	<code>GameManager.AddScore()</code>

레벨 진행

- 레벨마다 **LevelData** 에셋 1개
- 모든 벽돌 파괴 → 다음 레벨 로드
- 레벨별 벽돌 배치 / 공 속도 변경

생명 (Lives)

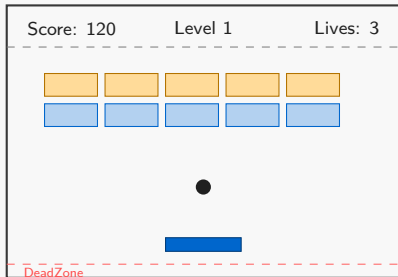
시작 **3개**(`GameSettings`) → 공 낙하 시 $-1 \rightarrow 0$ 이 되면 **Game Over**

난이도 (Difficulty)

GameSettings로 조절: 시작 생명, 패들 속도, 공 속도 배율(Easy/Normal/Hard)

화면 레이아웃과 데이터 설계

게임 화면 와이어프레임



HUD 요소

- 좌상단 **Score**, 우상단 **Lives**, 상단 중앙 **Level**
- **GameOverPanel** - 평소 숨김, 패배 시 표시

데이터 설계 (Data-Driven)

LevelData 벽돌 배치, 공 속도
GameSettings 생명, 패들 속도, 난이도

데이터를 **ScriptableObject**로 분리 → 코드 수정 없이 콘텐츠 추가

Paddle.cs – 키보드/마우스 입력

Paddle.cs (패들을 좌우로 이동)

```
1 using UnityEngine;
2
3 public class Paddle : MonoBehaviour
4 {
5     [SerializeField] float speed = 12f;    // 키보드 이동 속도
6     [SerializeField] float minX = -7.5f;   // 좌측 한계
7     [SerializeField] float maxX = 7.5f;    // 우측 한계
8     [SerializeField] bool useMouse = true; // 마우스키보드/ 전환
9
10    void Update()
11    {
12        float x = transform.position.x;
13
14        if (useMouse)
15        {
16            // 마우스 화면 좌표 -> 월드 좌표 변환
17            Vector3 world = Camera.main.ScreenToWorldPoint(Input.mousePosition);
18            x = world.x;
19        }
20        else
21        {
22            // 키보드: A/D, 좌우/ 화살표 (Horizontal 축)
23            float h = Input.GetAxisRaw("Horizontal");
24            x += h * speed * Time.deltaTime;
25        }
26
27        // 화면 밖으로 나가지 않도록 범위 제한
28        x = Mathf.Clamp(x, minX, maxX);
29        transform.position = new Vector3(x, transform.position.y, 0f);
30    }
31 }
```


Ball.cs – 발사와 일정 속도 유지

Ball.cs (좌클릭으로 발사, 물리로 튕김)

```
1 using UnityEngine;
2
3 [RequireComponent(typeof(Rigidbody2D))]
4 public class Ball : MonoBehaviour
5 {
6     [SerializeField] Paddle paddle;
7     [SerializeField] float launchSpeed = 8f;
8
9     Rigidbody2D rb;
10    Vector3 offset;    // 발사 전 패들과의 상대 위치
11    bool launched = false;
12
13    void Start()
14    {
15        rb = GetComponent<Rigidbody2D>();
16        offset = transform.position - paddle.transform.position;
17    }
18
19    void Update()
20    {
21        if (!launched)
22        {
23            // 발사 전: 패들을 따라다님
24            transform.position = paddle.transform.position + offset;
25            if (Input.GetMouseButtonDown(0)) Launch();    // 좌클릭으로 발사
26        }
27        else
28        {
29            // 충돌로 속도가 변해도 일정한 빠르기를 유지
30            rb.linearVelocity = rb.linearVelocity.normalized * launchSpeed;
31        }
32    }
33
34    void Launch()
35    {
36        launched = true;
37        Vector2 dir = new Vector2(Random.Range(-0.5f, 0.5f), 1f).normalized;
38        rb.linearVelocity = dir * launchSpeed;    // 구버전 는 Unity rb.velocity
39    }
40 }
```

[이론] 충돌 감지와 메시지(이벤트) 시스템

메시지 기반 충돌 통지

- 물리 엔진이 충돌을 감지하면, 해당 오브젝트의 스크립트에 **메시지(콜백)**를 보낸다
- 매 프레임 직접 검사할 필요 없이 **충돌 시점에만** 함수가 호출됨 (이벤트 기반)

이벤트가 호출되는 조건

- 두 오브젝트 모두 **Collider2D**가 있어야 함
- 적어도 **한쪽에 Rigidbody2D**가 있어야 함

Layer Collision Matrix

레이어(Layer)별로 어떤 레이어끼리 충돌할지 표로 설정
(Project Settings → Physics 2D).
불필요한 충돌 검사를 꺼서 **성능을 높이고**
의도한 충돌만 남긴다.

생명주기 콜백 (Enter/Stay/Exit)

Enter 닿기 시작한 순간 (1회)
Stay 닿아 있는 동안 (매 프레임)
Exit 떨어지는 순간 (1회)

충돌: OnCollision...2D, 트리거: OnTrigger...2D

충돌 이벤트 – Collision vs Trigger

OnCollisionEnter2D

두 콜라이더가 물리적으로 부딪칠 때 호출.

- 인자: Collision2D (접촉점, 상대 속도 등 포함)
- 볼이 벽돌/벽/패들에 튕기는 상황

OnTriggerEnter2D

한쪽 콜라이더가 **Is Trigger**일 때, 통과하면서 호출.

- 인자: Collider2D
- 볼이 **DeadZone**을 통과(낙하)하는 상황

이벤트가 호출되는 조건

- 두 오브젝트 모두 **Collider2D**가 있어야 함
- 적어도 **한쪽**에 **Rigidbody2D**가 있어야 물리 이벤트가 발생
- CompareTag()로 대상을 확인한 뒤 처리

설계 결정

벽돌 파괴	OnCollisionEnter2D
볼 낙하	OnTriggerEnter2D

Brick.cs – 충돌, 점수, 파괴

Brick.cs (볼에 맞으면 손상/파괴되고 점수 부여)

```
1 using UnityEngine;
2
3 [RequireComponent(typeof(SpriteRenderer))]
4 public class Brick : MonoBehaviour
5 {
6     [SerializeField] int hitPoints = 1;    // 파괴에 필요한 총돌 횟수
7     [SerializeField] int scoreValue = 10;  // 파괴 시 점수
8     [SerializeField] Sprite[] damageStates; // 손상 단계별 스프라이트
9
10    SpriteRenderer sr;
11    int currentHits = 0;
12
13    void Awake() { sr = GetComponent<SpriteRenderer>(); }
14
15    void OnCollisionEnter2D(Collision2D collision)
16    {
17        if (!collision.gameObject.CompareTag("Ball")) return; // 볼만 처리
18
19        currentHits++;
20        if (currentHits >= hitPoints) DestroyBrick();
21        else ShowDamage();
22    }
23
24    void ShowDamage()
25    {
26        int idx = currentHits - 1; // 손상 스프라이트로 교체
27        if (idx < damageStates.Length) sr.sprite = damageStates[idx];
28    }
29
30    void DestroyBrick()
31    {
32        GameManager.Instance.AddScore(scoreValue); // 점수 가산
33        GameManager.Instance.OnBrickDestroyed();    // 남은 벽돌 감소
34        Destroy(gameObject);
35    }
36 }
```

DeadZone.cs – 볼 낙하 감지 (트리거)

DeadZone.cs (화면 아래로 떨어진 볼을 감지)

```
1 using UnityEngine;
2
3 // 패들 아래에 배치한 가로로 긴 BoxCollider2D, Is Trigger = true
4 public class DeadZone : MonoBehaviour
5 {
6     void OnTriggerEnter2D(Collider2D other)
7     {
8         if (other.CompareTag("Ball"))
9         {
10             GameManager.Instance.LoseLife(); // 생명 1 감소 게임(오버 판정 포함)
11             // 볼을 패들 위로 되돌리거나 새 볼을 생성하는 처리...
12         }
13     }
14 }
```

동작 흐름

볼이 패들을 지나쳐 떨어지면 → DeadZone 트리거 진입 → GameManager가 생명을 줄이고, 생명이 0 이면 게임 오버 이벤트를 발생.

[이론] 프리팹(Prefab) – 개념과 원리

정의

완성된 GameObject의 컴포넌트 구성 전체를 재사용 가능한 에셋(원본 틀)으로 저장한 것.

원본(Asset)과 인스턴스(Instance)

- 씬에 배치된 복제본 = 인스턴스, 원본과 연결(link) 유지
- 원본 수정 → 모든 인스턴스에 자동 반영
- 오버라이드(Override) – 특정 인스턴스만 값 변경(굵게 표시), Apply/Revert로 관리

확장 기능

- 프리팹 변형(Variant) – 원본을 상속한 변형 버전(예: 빨강/파랑 벽돌)
- 중첩 프리팹(Nested) – 프리팹 안에 프리팹
- 프리팹 모드 – 격리된 편집 화면

장점

- 일관성 – 한 곳만 고치면 전체 반영
- 생산성 – 복제로 빠른 배치
- 동적 생성 – 런타임에 Instantiate()

프리팹 - 재사용 가능한 객체

프리팹이란?

- 컴포넌트 구성을 마친 GameObject를 **템플릿 (에셋)**으로 저장
- 씬에 여러 **인스턴스**로 배치 가능
- 원본 수정 → 모든 인스턴스에 **일괄 반영**

생성 방법

Hierarchy의 오브젝트를 **Project 창**으로 드래그
→ .prefab 에셋 생성

벽돌깨기의 프리팹 활용

프리팹	용도
Brick	레벨마다 수십 개 배치
Ball	멀티볼 파워업 시 복제
Paddle	씬 재사용
PowerUp	벽돌에서 드롭

코드로 생성

```
Instantiate(brickPrefab, pos,  
Quaternion.identity, bricksParent);
```

레벨 데이터를 읽어 벽돌을 코드로 자동 배치한다.

[이론] 스크립터블 객체(ScriptableObject) – 개념과 원리

정의

MonoBehaviour처럼 데이터를 담지만 **GameObject**에 붙지 않는 데이터 전용 클래스. ScriptableObject를 상속하고 .asset 파일로 저장된다.

MonoBehaviour vs ScriptableObject

	MonoBehaviour	ScriptableObject
부착 대상	GameObject	없음(에셋)
존재 위치	씬	프로젝트
인스턴스	오브젝트마다	에셋 1개 공유
용도	행동/로직	데이터 보관

핵심 특징

- 씬/오브젝트와 독립 - 프로젝트 에셋으로 존재
- 공유 참조 - 하나의 인스턴스를 여러 곳이 참조(데이터 중복 없음)
- 데이터 주도 설계 - 로직(코드)과 데이터(에셋) 분리
- 에디터에서 직관적 편집(비프로그래머도 가능)

용도와 한계

용도: 게임 설정, 레벨 데이터, 아이템/무기 스탯

한계: int[,] 2차원 배열 직렬화 불가; 에디터에서의 런타임 변경은 에셋에 남을 수 있음

ScriptableObject – 데이터 주도 설계

ScriptableObject란?

- 씬/게임오브젝트와 독립적인 데이터 컨테이너 에셋
- [CreateAssetMenu]로 우클릭 메뉴에서 생성
- 로직(코드)과 데이터(에셋)를 분리
- 에디터에서 직관적으로 값 편집

벽돌깨기 적용

LevelData – 레벨별 벽돌 배치/볼 속도

GameSettings – 시작 생명, 패들 속도, 난이도

GameManager

로직(코드)

데이터 참조

Scriptable Objects

LevelData.asset

GameSettings.asset

장점

코드 수정 없이 에셋만 바꿔 레벨/난이도를 추가할 수 있다.

LevelData.cs – 레벨 데이터(SO)

LevelData.cs

```
1 using UnityEngine;
2
3 [CreateAssetMenu(fileName = "LevelData",
4                   menuName = "Brick Breaker/Level Data")]
5 public class LevelData : ScriptableObject
6 {
7     [Header("레벨 정보")]
8     public string levelName = "Level 1";
9     public float ballSpeed = 8f;
10    public int timeLimit = 0;    // 0 = 무제한
11
12    [Header("벽돌 배치 행(단위)")]
13    public int columns = 10;
14    public BrickRow[] rows;    // 위에서 아래로 한 행씩
15 }
16
17 [System.Serializable]
18 public class BrickRow
19 {
20     // 0 = 빈칸, 1.. = 벽돌 종류프리맵(인덱스)
21     public int[] cells;
22 }
```

2차원 배열 주의

Unity 인스펙터는 int[,] 같은 2차원 배열을 직렬화하지 못한다.

그 래 서 한 행 을
[System.Serializable]
BrickRow 클래스로 감싸
배열의 배열 형태로 표현한다.

에셋 생성

Project 우클릭 →
Create → Brick Breaker →
Level Data

GameSettings.cs – 게임 설정(SO)

GameSettings.cs

```
1 using UnityEngine;
2
3 [CreateAssetMenu(fileName = "GameSettings",
4                 menuName = "Brick Breaker/Game Settings")]
5 public class GameSettings : ScriptableObject
6 {
7     [Header("게임 규칙")]
8     public int startingLives = 3;      // 시작 생명
9     public float paddleSpeed = 12f;    // 패들 이동 속도
10
11     [Header("난이도")]
12     [Range(0.5f, 2f)]
13     public float ballSpeedMultiplier = 1.0f; // 볼 속도 배율
14 }
```

설정을 에셋으로 분리하면

Easy / Normal / Hard 세 개의 GameSettings 에셋을 만들어 두고, GameManager에 어떤 에셋을 끼우느냐로 난이도를 즉시 바꿀 수 있다.

[이론] 싱글톤 패턴과 이벤트 기반 설계

싱글톤(Singleton) 패턴

- 특정 클래스의 인스턴스가 게임 전체에 단 하나만 존재
- static Instance 프로퍼티로 어디서나 접근
- Awake()에서 중복 인스턴스 제거
- 점수·생명 같은 전역 상태 관리에 적합

델리게이트와 이벤트(event)

- 델리게이트 - 메서드를 담는 변수.
Action<T>은 미리 정의된 델리게이트
- 발행-구독(Pub-Sub) - 발행자가 Invoke, 구독자가 +=로 구독
- 발행자는 구독자를 직접 모름 → 느슨한 결합

주의

전역 접근은 편하지만 남용하면 강한 결합이 생겨 유지보수가 어려워진다.

벽돌깨기 적용

GameManager가 점수/생명 변화를 발행하면 HUD가 구독해 UI를 갱신. UI를 바꿔도 게임 로직은 그대로다.

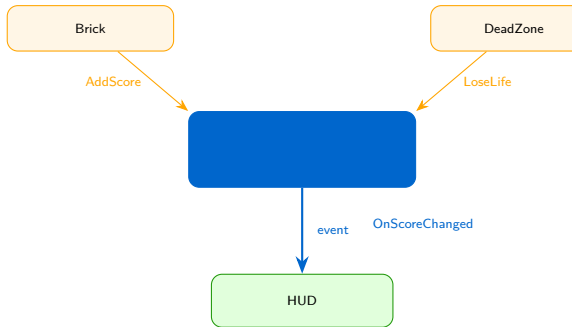
GameManager 설계 - 싱글톤 + 이벤트

역할

- 게임 상태의 단일 소유자: 점수, 생명, 남은 벽돌
- 싱글톤: `GameManager.Instance`로 어디서나 접근
- 상태가 바뀌면 **이벤트로 알림** (UI는 구독만)

느슨한 결합

GameManager는 HUD를 직접 알지 못한다.
Action 이벤트를 통해 통신하므로, UI를 바꿔도
게임 로직은 그대로다.



GameManager.cs – 핵심 구현

GameManager.cs (필드 / 생명주기 / 이벤트 메서드)

```
1 using UnityEngine;
2 using System;
3
4 public class GameManager : MonoBehaviour
5 {
6     public static GameManager Instance { get; private set; }
7
8     [SerializeField] GameSettings settings;    // SO 참조
9     [SerializeField] LevelData currentLevel;    // SO 참조
10
11     [Header("벽돌 배치")]
12     [SerializeField] Brick[] brickPrefabs;    // cells 값(1..N) -> brickPrefab값[-1]
13     [SerializeField] Transform bricksParent;    // 빈 오브젝트 "Bricks"
14     [SerializeField] Vector2 startPosition = new Vector2(-4.5f, 4.0f); // 좌상단 셀
15     [SerializeField] Vector2 cellSize = new Vector2(1.0f, 0.4f);    // 셀 간격
16
17     public int Score { get; private set; }
18     public int Lives { get; private set; }
19
20     // GameManager 구동하는 이벤트 느슨한(필합)
21     public event Action<int> OnScoreChanged;
22     public event Action<int> OnLivesChanged;
23     public event Action OnGameOver;
24
25     int remainingBricks;
26
27     void Awake()
28     {
29         if (Instance != null && Instance != this) { Destroy(gameObject); return; }
30         Instance = this;
31     }
32
33     void Start()
34     {
35         Lives = settings.startingLives;
36         SpawnBricks();    // 레벨 데이터로 벽돌 생성
37         OnScoreChanged?.Invoke(Score);
38         OnLivesChanged?.Invoke(Lives);
39     }
40
41     public void AddScore(int amount)
42     {
43         Score += amount;
44         OnScoreChanged?.Invoke(Score);    // 점수 변경 알림
45     }
46
47     public void OnBrickDestroyed()
48     {
49         remainingBricks--;
50         if (remainingBricks <= 0) Debug.Log("Level Clear!");
51     }
52
53     public void LoseLife()
54     {
55         Lives--;
56         OnLivesChanged?.Invoke(Lives);    // 생명 변경 알림
57         if (Lives <= 0) OnGameOver?.Invoke();    // 게임 모버 알림
58     }
59 }
```

GameManager.cs – SpawnBricks (레벨 데이터로 벽돌 생성)

SpawnBricks() – LevelData를 읽어 Instantiate

```
1 void SpawnBricks()
2 {
3     remainingBricks = 0;
4     if (currentLevel == null || currentLevel.rows == null) return;
5
6     int rowCount = currentLevel.rows.Length;
7     int colCount = currentLevel.columns;
8
9     // rows가 [0] 맨 위 행 -> 행 인덱스가 커질수록 는y 감소
10    for (int r = 0; r < rowCount; r++)
11    {
12        BrickRow row = currentLevel.rows[r];
13        if (row == null || row.cells == null) continue;
14
15        for (int c = 0; c < colCount && c < row.cells.Length; c++)
16        {
17            int cell = row.cells[c];
18            if (cell <= 0) continue;           // 0 = 빈칸
19            int idx = cell - 1;               // 1.. -> 프리팹 인덱스
20            if (idx >= brickPrefabs.Length) continue; // 안전장치
21
22            Vector2 pos = startPosition
23                + new Vector2(c * cellSize.x, -r * cellSize.y);
24            Instantiate(brickPrefabs[idx], pos, Quaternion.identity, bricksParent);
25            remainingBricks++;
26        }
27    }
28 }
```

HUD 구성 – Canvas와 UI 요소

Canvas (HUD) 구조

▽ Canvas (Screen Space - Overlay)

ScoreText (TMP) -- 좌상단

LivesText (TMP) -- 우상단

LevelText (TMP) -- 상단 중앙

▽ GameOverPanel (비활성)

Dark Background (Image)

"GAME OVER" (TMP)

Restart Button

EventSystem (자동 생성)

게임 오버 패널

- 평소엔 **SetActive(false)**로 숨김
- OnGameOver 이벤트 수신 시 **SetActive(true)**
- Restart 버튼 onClick으로 씬 재시작

UI와 로직의 분리

HUD는 표시만 담당하고, 점수/생명 계산은 GameManager가 담당. 역할이 명확히 나뉜다.

Anchor 활용

점수는 좌상단, 생명은 우상단 앵커로 고정 → 해상도가 변해도 위치 유지.

HUD.cs – 이벤트 구독으로 UI 갱신

HUD.cs

```
1 using UnityEngine;
2 using TMPro;
3
4 public class HUD : MonoBehaviour
5 {
6     [SerializeField] TextMeshProUGUI scoreText;
7     [SerializeField] TextMeshProUGUI livesText;
8     [SerializeField] GameObject gameOverPanel;
9
10    void Start() // Awake 이후라 GameManager.가Instance 준비되어 있음
11    {
12        GameManager.Instance.OnScoreChanged += UpdateScore;
13        GameManager.Instance.OnLivesChanged += UpdateLives;
14        GameManager.Instance.OnGameOver += ShowGameOver;
15    }
16
17    void OnDestroy() // 파괴 시 구독 해제 메모리(누수예외) 방지
18    {
19        if (GameManager.Instance == null) return;
20        GameManager.Instance.OnScoreChanged -= UpdateScore;
21        GameManager.Instance.OnLivesChanged -= UpdateLives;
22        GameManager.Instance.OnGameOver -= ShowGameOver;
23    }
24
25    void UpdateScore(int s) => scoreText.text = $"Score: {s}";
26    void UpdateLives(int l) => livesText.text = $"Lives: {l}";
27    void ShowGameOver() => gameOverPanel.SetActive(true);
28 }
```

WebGL 빌드 설정

플랫폼 전환

- 1 **File** → **Build Profiles** (구버전: Build Settings)
- 2 플랫폼 목록에서 **Web** 선택
- 3 **Switch Platform** 클릭
- 4 **Build** → 출력 폴더 지정

빌드 산출물

index.html, Build/, TemplateData/
→ 이 폴더 전체를 정적 호스팅하면 실행된다.

압축(Compression) 주의

Player Settings → Publishing Settings:

- GitHub Pages는 서버 헤더를 설정할 수 없다.
- **Compression Format = Disabled** 또는
- Gzip/Brotli 사용 시 **Decompression Fallback**을 체크

그렇지 않으면 브라우저에서 로딩 실패(빈 화면)한다.

참고

배포 세부 절차는 **Lec11_Deploy** 자료를 함께 참고.

GitHub Pages로 배포하기

배포 절차

- 1 GitHub에 저장소 생성, 코드 push
- 2 WebGL 빌드 결과를 docs/ 폴더에 넣고 commit
- 3 **Settings** → **Pages**
- 4 Source: main 브랜치 / /docs 폴더 선택
- 5 발급된 URL에서 게임 실행 확인

배포 명령 예시

```
# 빌드 결과를 docs/ 에 두고
git add docs
git commit -m "Add WebGL build"
git push origin main
# Settings > Pages 에서
#   main / docs 선택 -> 게시
```

접속 주소

[https://사용자명.github.io/
저장소명/](https://사용자명.github.io/저장소명/)

단계별 구현 순서

순차적으로 쌓아 올리기 – 매 단계가 플레이 가능

각 단계가 끝날 때마다 게임이 한 단계씩 더 동작하도록 만든다. 큰 그림을 한 번에 짜지 말 것.

1~3단계: 핵심 플레이

- ❶ 패들 + 볼 물리 – 입력으로 패들 이동, 볼 발사/반사
- ❷ 벽돌 충돌 감지 – OnCollisionEnter2D로 파괴
- ❸ 게임 매니저 + 점수 – 싱글톤, 점수/생명

4~6단계: 완성도와 배포

- ❹ HUD UI – 점수/생명/레벨 표시, 게임 오버
- ❺ 프리팹 & SO 리팩토링 – 재사용/데이터 분리
- ❻ WebGL 빌드 & 배포 – 깃헙 Pages 호스팅

도전 과제

기본 확장

- 여러 레벨 – LevelData 에셋을 여러 개 만들어 클리어 시 다음 레벨 로드
- 내구도 벽돌 – hitPoints가 2 이상인 벽돌과 손상 스프라이트
- 사운드 – 충돌/파괴/게임 오버 효과음

심화 도전

- 파워업 – 벽돌에서 드롭되는 멀티볼/패들 확장 아이템 (프리팸)
- 최고 점수 – PlayerPrefs로 저장
- 일시정지 – Esc 키로 Time.timeScale 제어

전체 구조 및 스크립트 연결

Assets 폴더 구조

```
Assets/  
  Scripts/  
    Paddle.cs, Ball.cs  
    Brick.cs, DeadZone.cs  
    GameManager.cs, HUD.cs  
    LevelData.cs, GameSettings.cs  
  Prefabs/  
    Brick.prefab, Ball.prefab  
    Paddle.prefab  
  Data/  
    Level1.asset, Level2.asset  
    Normal.asset (GameSettings)  
  Sprites/, Materials/  
  Scenes/ -- BrickBreaker.unity
```

스크립트 요약

스크립트	역할
Paddle.cs	키보드/마우스로 패들 이동
Ball.cs	발사, 일정 속도 유지 (물리)
Brick.cs	충돌 감지, 손상/파괴, 점수
DeadZone.cs	볼 낙하 감지 (트리거)
GameManager.cs	점수/생명/상태, 이벤트 발생
HUD.cs	이벤트 구독, UI 갱신
LevelData.cs	벽돌 배치 데이터 (SO)
GameSettings.cs	규칙/난이도 데이터 (SO)

학습 내용 요약

이번 강의에서 배운 것

- 하이ера키 - 부모로 묶어 구조화
- 프리팹 - 벽돌/볼 재사용
- **ScriptableObject** - 데이터 분리
- **Rigidbody2D / Collider2D** - 물리
- 충돌/트리거 이벤트 - 게임 로직
- 키보드/마우스 입력 - 패들/발사
- 싱글톤 + 이벤트 - 상태 관리
- **HUD** - UI와 로직의 분리
- **WebGL 빌드 + 깃헙 Pages** - 배포

핵심 설계 패턴

- ① **데이터 분리**: ScriptableObject로 레벨/설정을 코드 밖으로
- ② **느슨한 결합**: 이벤트로 GameManager→HUD 통신
- ③ **단일 책임**: 각 스크립트가 한 가지 일만
- ④ **점진적 구현**: 항상 플레이 가능한 상태 유지

마무리

한 학기의 모든 개념이 하나의 게임으로 모였습니다.
이제 여러분만의 게임을 만들어 보세요! 🚀